TOP LEARNING · INTELIGENCIA ARTIFICIAL Y AUTOMATIZACIÓN CON N8N

VIBE CODING

Guion docente de la presentación «De una idea escrita a una aplicación funcional». Clase en vivo · 27 de octubre de 2026 · 25 minutos.

> El guion está escrito a 65–120 palabras por minuto para dejar margen a preguntas, pausas y animaciones. Teclas: → avanza · ← retrocede · P vista de presentador · N notas · G índice · F pantalla completa · L modo ligero.

#	Diapositiva	Tiempo	Acumulado
1	VIBE CODING: el software cambia de paradigma	1:00	1:00
2	El problema antes de la IA	1:30	2:30
3	¿Qué es exactamente el vibe coding?	2:30	5:00
4	¿Cómo funciona?	2:00	7:00
5	Qué sucede detrás de una instrucción	2:00	9:00
6	Tres formas de desarrollar software	2:00	11:00
7	Herramientas del ecosistema	1:30	12:30
8	De un prompt básico a una especificación útil	2:00	14:30
9	Caso práctico: dashboard financiero empresarial	2:30	17:00
10	Cómo comprobar que nuestra aplicación funciona	1:30	18:30
11	Aplicaciones reales en empresas	1:30	20:00
12	Riesgos y limitaciones	1:30	21:30
13	De prototipo a producto profesional	1:00	22:30
14	El ejercicio de hoy	1:30	24:00
15	Conclusión	1:00	25:00
	Total	**25:00**	



DIAPOSITIVA 1

VIBE CODING: el software cambia de paradigma

1:00

GUION ORAL

Bienvenidas y bienvenidos. Hoy vamos a hablar de un término que en 2025 pasó de una publicación en redes sociales a ser elegido palabra del año: vibe coding.

Empiezo con una pregunta: ¿y si pudieras construir una aplicación simplemente describiendo lo que necesitas? Hace pocos años sonaba a ciencia ficción. Hoy es posible en parte, y digo «en parte» a propósito: al terminar sabrán qué es real, qué requiere trabajo y qué sigue siendo nuestra responsabilidad.

En la imagen está el viaje de hoy: a un lado, una idea, orgánica y difusa; al otro, software, estructurado en capas. Lo que ocurre en medio es lo que vamos a entender, y después lo vamos a practicar con un dashboard financiero.

EXPLICACIÓN TÉCNICA

Todavía no hace falta explicar nada técnico. El objetivo es fijar expectativas: veremos el concepto, cómo funciona, sus límites y una práctica guiada.

PREGUNTA DE PARTICIPACIÓN

> Escriban en el chat: ¿alguna vez han necesitado en su trabajo una herramienta que no existía o que nadie tenía tiempo de construir?

TRANSICIÓN

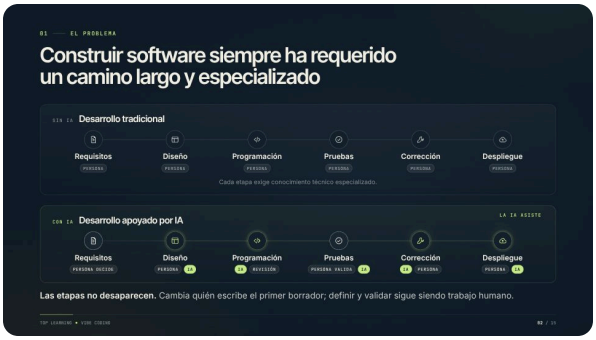
Para entender por qué esto es un cambio de paradigma, primero veamos cómo se construía software antes de la IA.

ERRORES CONCEPTUALES QUE EVITAR

- No prometas que «cualquiera hará cualquier aplicación sin saber nada». La promesa correcta: hoy entenderán cómo funciona y construirán un prototipo verificado.

FUENTES

- Karpathy, A. (2 de febrero de 2025). Publicación en X que acuña «vibe coding». — https://x.com/karpathy/status/1886192184808149383
- Collins Dictionary. «vibe coding», palabra del año 2025 (anuncio del 6 de noviembre de 2025; cobertura de I Programmer). — https://www.i-programmer.info/news/85-humour-/18446-vibe-coding-is-collins-word-of-the-year-2025.html



DIAPOSITIVA 2

El problema antes de la IA

1:30

GUION ORAL

Crear software, tradicionalmente, es un ciclo con etapas especializadas. Primero se definen los requisitos: qué debe hacer el sistema y bajo qué reglas. Luego se diseña la interfaz, se programa, se prueba, se corrigen errores y se despliega, es decir, se publica para que otras personas la usen.

Cada etapa requiere conocimientos que toma años dominar. Por eso, cuando alguien de Finanzas o de Recursos Humanos necesitaba una herramienta a la medida, dependía de un equipo de desarrollo, de un presupuesto y de una fila de prioridades.

Ahora miren la segunda pista. Con IA, las etapas no desaparecen. Lo que cambia es que la IA puede escribir el primer borrador del código, proponer diseños y ayudar a encontrar y corregir errores. Pero alguien tiene que decidir qué se necesita y comprobar que el resultado es correcto. Esas dos partes siguen siendo humanas.

EXPLICACIÓN TÉCNICA

Requisitos: descripción precisa de lo que el sistema debe hacer. Desplegar (deploy): publicar la aplicación en un servidor o servicio para que funcione fuera de tu computadora.

ANALOGÍA

Construir una casa: el plano, la obra, la inspección y la entrega siguen existiendo aunque cambies las herramientas manuales por eléctricas. La herramienta acelera la obra, pero no sustituye al arquitecto ni a la inspección.

PREGUNTA DE PARTICIPACIÓN

> ¿Cuál de estas seis etapas creen que es la más difícil para alguien que no programa? Muchas personas dirán «programar». Ayúdales a ver que definir requisitos y validar también son difíciles, y que esas no las resuelve la IA por sí sola.

TRANSICIÓN

Entonces, ¿qué es exactamente eso que llamamos vibe coding?

ERRORES CONCEPTUALES QUE EVITAR

- No digas que la IA «reduce el desarrollo de meses a minutos» ni cites porcentajes de productividad: no hay una cifra general confiable y el efecto depende mucho del contexto.
- No desvalorices el desarrollo profesional; en la diapositiva 13 veremos por qué sigue siendo indispensable.



DIAPOSITIVA 3

¿Qué es exactamente el vibe coding?

2:30

GUION ORAL

El término lo acuñó Andrej Karpathy, investigador de inteligencia artificial, cofundador de OpenAI y exdirector de IA en Tesla, en una publicación en X el 2 de febrero de 2025.

Escribió, y lo traduzco: «Hay un nuevo tipo de programación al que llamo vibe coding, en el que te entregas por completo a las vibras, abrazas lo exponencial y olvidas que el código siquiera existe».

En la misma publicación contó que aceptaba todos los cambios de la IA sin leerlos («Accept All»), que pegaba los mensajes de error para que la IA los arreglara y que esto «no está tan mal para proyectos desechables de fin de semana». Esa es la clave del sentido original: experimentar con software generado por IA sin revisar el código a fondo, en proyectos donde equivocarse cuesta poco.

El término se volvió tan popular que Collins Dictionary lo eligió palabra del año 2025. Al popularizarse, también se amplió: hoy, en 2026, mucha gente dice vibe coding para referirse a cualquier desarrollo guiado por lenguaje natural con IA, aunque sí se revise y pruebe el código.

Simon Willison, desarrollador muy citado en este tema, insistió desde marzo de 2025 en no confundirlos: si revisaste, probaste y entiendes el código, eso no es vibe coding en el sentido original; es programación asistida por IA. En esta clase usaremos ambos sentidos, pero siempre diremos cuál.

EXPLICACIÓN TÉCNICA

«Accept All» significa aplicar los cambios propuestos por la herramienta sin leer el diff, es decir, la lista de líneas agregadas o eliminadas. En un proyecto profesional, esos cambios se revisan antes de integrarse.

ANALOGÍA

Es como pagar una cuenta sin revisar el ticket: en una comida casual da igual; en la contabilidad de la empresa, revisar cada cargo es obligatorio.

PREGUNTA DE PARTICIPACIÓN

> Si una persona usa IA para escribir el código, pero lo revisa línea por línea y lo prueba, ¿está haciendo vibe coding en el sentido original? Respuesta: no; es programación asistida por IA.

TRANSICIÓN

Ya sabemos qué es. Ahora veamos cómo funciona, paso a paso.

ERRORES CONCEPTUALES QUE EVITAR

- Karpathy no inventó la programación con IA: acuñó y popularizó el término.

- El sentido original no es «programar sin saber nada»: describe no revisar el código, en proyectos de bajo riesgo.
- Aclara que la cita es una traducción tuya y que el original está en inglés.

RIGOR: HECHO E INTERPRETACIÓN

Hechos documentados: autor, fecha y contenido de la publicación (2-feb-2025); distinción de Willison (19-mar-2025); palabra del año Collins 2025. Interpretación: el «uso amplio» del término en 2026 describe cómo se usa hoy, no una definición oficial.

FUENTES

- Karpathy, A. (2 de febrero de 2025). Publicación en X que acuña «vibe coding». — https://x.com/karpathy/status/1886192184808149383
- Willison, S. (19 de marzo de 2025). «Not all AI-assisted programming is vibe coding (but vibe coding rocks)». — https://simonwillison.net/2025/Mar/19/vibe-coding/
- Fowler, M. «Vibe Coding» (bliki). Uso adecuado y riesgos. — https://martinfowler.com/bliki/VibeCoding.html
- Collins Dictionary. «vibe coding», palabra del año 2025 (anuncio del 6 de noviembre de 2025; cobertura de I Programmer). — https://www.i-programmer.info/news/85-humour-/18446-vibe-coding-is-collins-word-of-the-year-2025.html



DIAPOSITIVA 4

¿Cómo funciona?

2:00

GUION ORAL

Recorramos el ciclo. Uno, la idea: todo empieza con un problema real, por ejemplo, «pierdo dos horas cada lunes armando el reporte de flujo». Dos, la instrucción: lo describes en lenguaje natural; mientras más clara, mejor. Tres, el modelo de IA interpreta tu instrucción y propone una solución; si es un agente, también arma un plan.

Cuatro, código: la IA escribe o modifica archivos. Cinco, ejecución: la aplicación se pone a funcionar, la abre el agente o la abres tú. Seis, pruebas: compruebas si hace lo que pediste; algunas herramientas corren pruebas automáticas, pero la verificación final es tuya. Siete, iteración: describes lo que falta o falló y vuelves al paso dos.

Fíjense en los colores: marfil es lo que hace la persona; verde lima, lo que hace la IA. La persona está al inicio y al final de cada vuelta. Y casi nunca sale perfecto a la primera: el valor está en el ciclo.

EXPLICACIÓN TÉCNICA

Un agente de programación es un modelo de lenguaje conectado a herramientas: puede leer archivos del proyecto, editarlos, ejecutar comandos y ver los resultados, en un ciclo, hasta completar la tarea o pedir tu aprobación. Por eso puede «probar y corregir» por sí mismo, dentro de los permisos que le des.

ANALOGÍA

Como trabajar con un asistente muy rápido que no conoce tu empresa: le explicas, hace un borrador, lo revisas y le das retroalimentación. Cada vuelta mejora el resultado.

PREGUNTA DE PARTICIPACIÓN

> ¿En qué paso creen que se cometen más errores? Lo esperable: en la instrucción, por lo que no se dice, y en las pruebas, por lo que no se revisa.

TRANSICIÓN

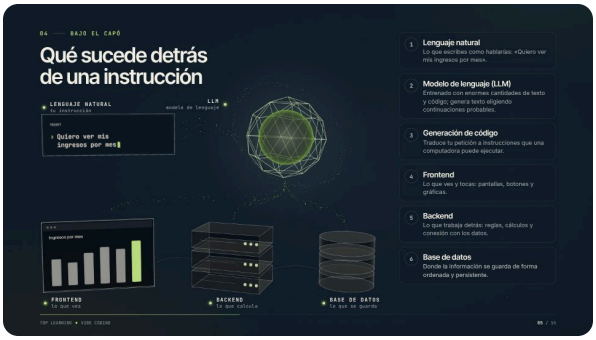
Pero ¿qué ocurre realmente en los pasos tres y cuatro? Veamos qué hay detrás de una instrucción.

ERRORES CONCEPTUALES QUE EVITAR

- No digas que la IA «entiende tu negocio»: solo conoce lo que le describes y lo que hay en el proyecto.
- No presentes el ciclo como lineal y de una sola vuelta: lo normal son varias iteraciones.

FUENTES

- Microsoft. «Best practices for using AI in VS Code». Documentación de Visual Studio Code. — https://code.visualstudio.com/docs/agents/best-practices



DIAPOSITIVA 5

Qué sucede detrás de una instrucción

2:00

GUION ORAL

Uno, lenguaje natural: lo que escribes tal como hablarías, por ejemplo, «quiero ver mis ingresos por mes».

Dos, el modelo de lenguaje, o LLM: un modelo entrenado con enormes cantidades de texto, incluido código. Genera su respuesta pieza por pieza, eligiendo continuaciones probables según el contexto. Tres, generación de código: así traduce tu petición a instrucciones que una computadora puede ejecutar.

Ese código suele repartirse en tres partes. Cuatro, el frontend: lo que ves y tocas, como pantallas, botones y gráficas. Cinco, el backend: lo que trabaja detrás, como reglas, cálculos y conexión con los datos. Seis, la base de datos: donde la información se guarda de forma ordenada y persistente.

Un detalle que usaremos en la práctica: por seguridad, una página web no puede leer archivos de tu computadora sin que tú los selecciones. Por eso, para leer un Excel de forma automática, necesitaremos un backend.

EXPLICACIÓN TÉCNICA

Un LLM (Large Language Model) genera texto «token por token», es decir, fragmentos de palabras, a partir de patrones aprendidos. Optimiza lo plausible, no lo verdadero: por eso puede escribir código que parece correcto y no lo es. En una aplicación web típica, el frontend corre en el navegador (HTML, CSS y JavaScript), el backend corre en un servidor o en tu computadora y la base de datos guarda la información de forma persistente.

ANALOGÍA

Un restaurante: el frontend es el comedor y el menú; el backend, la cocina; la base de datos, la despensa. El LLM sería un cocinero muy rápido que prepara recetas a partir de lo que pides, pero que a veces inventa ingredientes.

PREGUNTA DE PARTICIPACIÓN

> En la app de su banco, ¿qué parte es frontend y qué parte es backend?

TRANSICIÓN

Con estas piezas claras, comparemos tres maneras de construir software.

ERRORES CONCEPTUALES QUE EVITAR

- No digas que el LLM «copia» código de internet: genera texto nuevo a partir de patrones. Algunas herramientas además pueden buscar en la web, pero es una función aparte.

- No digas que el modelo «sabe» si su código funciona: solo lo sabe si se ejecuta y se observa el resultado.



DIAPOSITIVA 6

Tres formas de desarrollar software

2:00

GUION ORAL

Pongamos tres formas de trabajar en un mapa. Eje horizontal: quién escribe la mayor parte del código. Eje vertical: cuánto revisa y prueba una persona. Es un esquema conceptual, no datos medidos.

Programación tradicional: una persona escribe y revisa. Máximo control y comprensión, pero requiere formación y tiempo. Programación asistida por IA: la IA escribe buena parte del código, y una persona revisa, prueba y entiende lo que se integra. Vibe coding en su sentido original: la IA escribe y casi nadie revisa. Es lo más rápido para explorar una idea y lo más arriesgado para algo que importa.

Ninguna es «la correcta» en abstracto: depende de lo que está en juego. Para un prototipo personal de fin de semana, el vibe coding está bien. Para el sistema de nómina, no.

EXPLICACIÓN TÉCNICA

Revisión de código: leer los cambios para entender qué hacen y detectar errores. Pruebas: ejecutar el programa con casos conocidos para comprobar resultados, ya sea de forma manual o automatizada.

ANALOGÍA

Un contrato: (1) lo redacta un abogado; (2) la IA hace un borrador y el abogado lo revisa; (3) firmas el borrador de la IA sin leerlo. La tercera opción puede funcionar… hasta que no.

PREGUNTA DE PARTICIPACIÓN

> ¿Dónde ubicarían el ejercicio de hoy? Lo ideal: generar con IA y verificar el cálculo, es decir, moverse hacia la programación asistida.

TRANSICIÓN

¿Y con qué herramientas se hace todo esto? Veamos el ecosistema.

ERRORES CONCEPTUALES QUE EVITAR

- No presentes el vibe coding como algo «malo»: es útil y legítimo para explorar, aprender y prototipar.
- No inventes porcentajes en los ejes: el mapa es cualitativo.

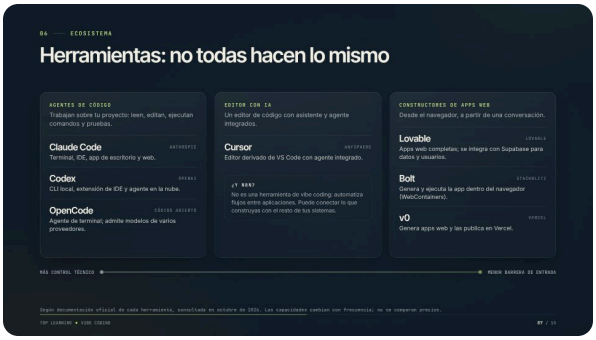
RIGOR: HECHO E INTERPRETACIÓN

Hecho documentado: la distinción entre vibe coding y programación asistida por IA (Willison, 2025; Fowler). Interpretación: la ubicación en el mapa es un esquema didáctico, no una medición.

FUENTES

- Willison, S. (19 de marzo de 2025). «Not all AI-assisted programming is vibe coding (but vibe coding rocks)». — https://simonwillison.net/2025/Mar/19/vibe-coding/

- Fowler, M. «Vibe Coding» (bliki). Uso adecuado y riesgos. — https://martinfowler.com/bliki/VibeCoding.html



DIAPOSITIVA 7

Herramientas del ecosistema

1:30

GUION ORAL

Estas herramientas no son equivalentes; las agrupo en tres familias. Agentes de código, como Claude Code, Codex y OpenCode: trabajan sobre un proyecto, leen archivos, los editan y ejecutan comandos y pruebas. Se usan desde la terminal, desde un editor o, en algunos casos, en la nube.

Editor con IA, como Cursor: un editor de código con asistente y agente integrados. Constructores de aplicaciones web, como Lovable, Bolt y v0: desde el navegador y a partir de una conversación, generan y publican aplicaciones web. Tienen la barrera de entrada más baja, a cambio de menos control sobre cómo y dónde corre la aplicación.

Las capacidades cambian casi cada mes; lo que ven aquí está basado en su documentación oficial a octubre de 2026. ¿Y n8n? Es una plataforma de automatización de flujos: puede ser el «pegamento» que conecte lo que construyan con otras aplicaciones.

EXPLICACIÓN TÉCNICA

Terminal (línea de comandos): interfaz de texto para dar instrucciones a la computadora. CLI: programa que se usa desde la terminal. IDE: editor de código con herramientas integradas, como VS Code. Detalle para la práctica: un constructor web corre en la nube y no puede leer un archivo de tu computadora; para el ejercicio con Excel local conviene un agente o un editor que trabaje en tu equipo.

ANALOGÍA

Agente de código: un colaborador que trabaja dentro de tu taller con tus herramientas. Editor con IA: tu propio taller con un asistente al lado. Constructor web: un taller rentado que ya trae todo, pero en el que tú no decides cómo está organizado.

PREGUNTA DE PARTICIPACIÓN

> ¿Alguien ya usó alguna de estas herramientas? ¿Para qué?

TRANSICIÓN

Sin importar la herramienta, hay algo que define la calidad del resultado: cómo pedimos las cosas.

ERRORES CONCEPTUALES QUE EVITAR

- No digas que una herramienta es «la mejor» en general: depende del caso, del presupuesto y del nivel técnico.
- No cites precios ni límites de uso: cambian con frecuencia. Si te preguntan, consulta la página oficial en vivo.

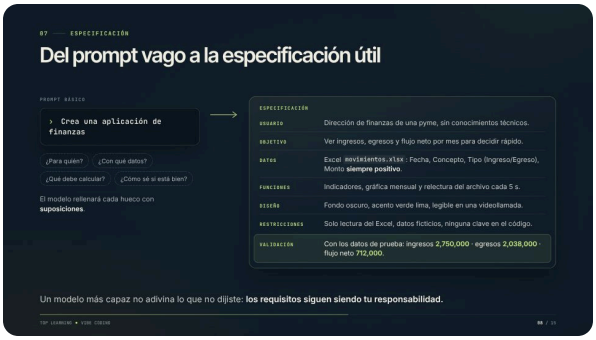
- No presentes n8n como una herramienta de vibe coding: es una plataforma de automatización de flujos, aunque incluye funciones de IA.

RIGOR: HECHO E INTERPRETACIÓN

Hechos documentados: descripción general de cada herramienta según su documentación oficial (consultada en octubre de 2026). No se comparan precios, rendimiento ni cuotas de mercado.

FUENTES

- Anthropic. Claude Code — documentación. — https://code.claude.com/docs/en/overview
- OpenAI. Codex — documentación. — https://developers.openai.com/codex
- OpenCode — documentación (proyecto de código abierto). — https://opencode.ai/docs
- Anysphere. Cursor — documentación. — https://docs.cursor.com
- Lovable — documentación. — https://docs.lovable.dev
- StackBlitz. Bolt — centro de soporte. — https://support.bolt.new
- Vercel. v0 — documentación. — https://v0.app/docs



DIAPOSITIVA 8

De un prompt básico a una especificación útil

2:00

GUION ORAL

A la izquierda, el prompt que casi todos escribimos la primera vez: «Crea una aplicación de finanzas». ¿Para quién? ¿Con qué datos? ¿Qué debe calcular? ¿Cómo sabremos si está bien? El modelo va a rellenar esos huecos con suposiciones.

A la derecha, la misma petición convertida en especificación, con siete campos. Usuario: para quién es. Objetivo: qué decisión ayuda a tomar. Datos: de dónde salen y con qué estructura. Funciones: qué debe hacer. Diseño: cómo debe verse. Restricciones: qué no debe hacer. Y, el más importante, criterios de validación: con estos datos de prueba, el flujo neto debe dar 712 mil pesos.

Aunque los modelos sean cada año más capaces, no pueden adivinar lo que no dijiste: quién usará la aplicación, de dónde salen los datos ni cuáles son las reglas del negocio. Definir bien los requisitos sigue siendo nuestro trabajo.

EXPLICACIÓN TÉCNICA

Una especificación es la descripción verificable de lo que debe hacer el software. Los criterios de validación convierten «que funcione» en algo comprobable. Las guías de buenas prácticas para agentes, como la de VS Code, recomiendan acotar la tarea, dar contexto y revisar y validar el resultado antes de integrarlo.

ANALOGÍA

Pedir un pastel: «hazme un pastel» frente a «pastel de chocolate para 20 personas, sin nuez, para el sábado a las 5». El pastelero es igual de bueno; el resultado depende de tu pedido.

PREGUNTA DE PARTICIPACIÓN

> Para su propia área, ¿qué dato o regla del negocio tendrían que incluir sí o sí en la especificación?

TRANSICIÓN

Esa especificación es justamente la de nuestro caso práctico. Veámoslo funcionando.

ERRORES CONCEPTUALES QUE EVITAR

- No confundas «prompt largo» con «buen prompt»: lo que importa es que sea específico y verificable.
- Nunca pongas datos reales de la empresa, contraseñas ni claves de acceso en un prompt de práctica: usa datos ficticios.

FUENTES

- Microsoft. «Best practices for using AI in VS Code». Documentación de Visual Studio Code. — https://code.visualstudio.com/docs/agents/best-practices



DIAPOSITIVA 9

Caso práctico: dashboard financiero empresarial

2:30

GUION ORAL

Nuestra empresa ficticia, Comercializadora Nébula, quiere ver sus ingresos, egresos, flujo neto y evolución mensual. La arquitectura tiene cuatro piezas. El archivo de Excel con los movimientos. El backend, un pequeño programa que lee el archivo y calcula totales. La API, la «ventanilla» por la que el backend entrega esos resultados. Y el dashboard, que los muestra.

Simulemos un cambio. En Excel modifico el gasto en proveedores de junio: de 172 mil a 202 mil pesos, y guardo. Observen: el dashboard vuelve a pedir datos a la API, el backend relee el archivo y la respuesta trae los nuevos valores. Los egresos de junio pasan de 372 mil a 402 mil, y el flujo neto total, de 712 mil a 682 mil.

Ahora, lo más importante de esta diapositiva: esto ocurre porque está programado. Pedirle a la IA «que se actualice solo» no garantiza nada. Puede implementarlo bien, mal o de forma frágil. Hay que saber qué mecanismo se usó y comprobar que funciona.

EXPLICACIÓN TÉCNICA

API: interfaz con reglas para pedir datos; el dashboard solicita /api/resumen y recibe JSON. Hay tres formas comunes de actualizar: (1) relectura periódica (polling), en la que el dashboard pide datos cada pocos segundos; (2) vigilancia del archivo, en la que el backend detecta que cambió y avisa; (3) un botón «Actualizar». Detalles prácticos: hay que guardar el archivo, porque escribir la cifra no basta; si el Excel está en OneDrive o SharePoint, la copia local puede tardar en sincronizar, y leerlo directo de la nube requiere otra integración, como Microsoft Graph.

ANALOGÍA

Un tablero de vuelos en el aeropuerto: solo cambia si el sistema consulta la información y actualiza la pantalla. Si nadie consulta, el tablero sigue mostrando el dato viejo aunque el vuelo ya cambió.

PREGUNTA DE PARTICIPACIÓN

> Si cambio la cifra en Excel pero no guardo, ¿qué debería mostrar el dashboard? Respuesta: lo anterior.

TRANSICIÓN

Pero que el número cambie no significa que esté bien calculado. ¿Cómo comprobamos que nuestra aplicación funciona?

ERRORES CONCEPTUALES QUE EVITAR

- No digas que el dashboard está «conectado en vivo» a Excel por arte de magia: siempre hay un mecanismo de lectura.

- No confundas «se actualizó» con «calculó bien».
- La animación es una simulación didáctica, no la aplicación real del ejercicio.



DIAPOSITIVA 10

Cómo comprobar que nuestra aplicación funciona

1:30

GUION ORAL

El ciclo de validación tiene cinco pasos: generar, ejecutar, detectar errores, corregir y volver a probar.

Ejemplo real de lo que puede pasar. La aplicación se ve impecable y dice: flujo neto, 4 millones 788 mil pesos. Hacemos la prueba de control: ingresos, 2 millones 750 mil, menos egresos, 2 millones 38 mil. Da 712 mil. ¿Qué pasó? En el Excel, los egresos venían capturados con signo negativo, y el código los volvió a restar: menos por menos da más.

La corrección no es solo arreglar esa línea: es escribir en la especificación la convención de signos y agregar una prueba con el resultado esperado. Una interfaz bonita no garantiza cálculos correctos.

EXPLICACIÓN TÉCNICA

Prueba de control: calcular el resultado esperado por un camino independiente, por ejemplo con una calculadora o con SUMAR.SI.CONJUNTO en Excel, y compararlo. Mejor aún si se convierte en una prueba automática: un pequeño programa que verifica que, con los datos de prueba, el flujo neto sea 712,000.

ANALOGÍA

Una báscula digital descalibrada: la pantalla es nítida, pero el número está mal. Por eso se calibra con un peso conocido.

PREGUNTA DE PARTICIPACIÓN

> Si el dashboard dice 4,788,000 y su cálculo manual dice 712,000, ¿qué revisarían primero?

TRANSICIÓN

Con estas precauciones, ¿qué tipo de aplicaciones pueden construir las empresas?

ERRORES CONCEPTUALES QUE EVITAR

- No digas que la IA «siempre» o «nunca» se equivoca.
- No uses «pídele a la IA que revise si está bien» como única validación: valida con datos conocidos.

FUENTES

- Microsoft. «Best practices for using AI in VS Code». Documentación de Visual Studio Code. — https://code.visualstudio.com/docs/agents/best-practices



DIAPOSITIVA 11

Aplicaciones reales en empresas

1:30

GUION ORAL

Cuatro escenarios frecuentes. Un dashboard de ventas, que sustituye el reporte que alguien arma a mano cada semana. Un inventario básico, para saber qué hay en almacén. Una landing page con formulario, para captar prospectos de una campaña. Y una herramienta interna de seguimiento de clientes, para que ninguno se pierda entre correos.

Los cuatro se pueden prototipar rápido con IA. Pero fíjense en la etiqueta «necesita además»: para funcionar todos los días, la mayoría requiere piezas extra, como una base de datos persistente, inicio de sesión, respaldos, hosting o un destino para los datos del formulario, que puede ser un CRM, una hoja de cálculo o un webhook de n8n.

EXPLICACIÓN TÉCNICA

Persistencia: que los datos se conserven aunque cierres la aplicación o reinicies el servidor; un prototipo que guarda datos en la memoria o en el navegador los pierde o los deja solo en ese equipo. Autenticación: verificar quién eres; permisos: qué puedes hacer. Si un formulario recaba datos personales en México, se requiere un aviso de privacidad conforme a la legislación de protección de datos; confirma los detalles con el área legal.

ANALOGÍA

Una maqueta frente a una casa habitada: la maqueta muestra cómo se verá; la casa necesita cimientos, instalaciones y cerraduras.

PREGUNTA DE PARTICIPACIÓN

> ¿Cuál de estos cuatro escenarios le serviría más a su área de trabajo?

TRANSICIÓN

Muchas de estas aplicaciones funcionan perfecto en la demo. Veamos lo que la demo no muestra.

ERRORES CONCEPTUALES QUE EVITAR

- No digas que basta con el prototipo para operar: los cuatro casos necesitan piezas adicionales para el uso diario.
- No des asesoría legal específica sobre protección de datos: menciona la obligación general y remite al área legal.



DIAPOSITIVA 12

Riesgos y limitaciones

1:30

GUION ORAL

Arriba de la línea, lo que se ve: la demo funciona. Debajo, lo que no se ve.

Errores de lógica, como el de los signos que acabamos de ver. Vulnerabilidades de seguridad: formularios que aceptan cualquier cosa o rutas sin control de acceso. Exposición de información sensible: claves de acceso escritas en el código que llega al navegador. Dependencias o configuraciones incorrectas: un estudio presentado en USENIX Security 2025 documentó que los modelos a veces sugieren paquetes de software que no existen, y alguien malintencionado puede publicar uno con ese nombre. Código difícil de mantener, que nadie entiende ni se atreve a modificar. Y aplicaciones que funcionan en la demo, pero fallan con más usuarios, datos inesperados o una conexión lenta.

El principio: que funcione una vez no significa que esté listo para producción.

EXPLICACIÓN TÉCNICA

Clave de API: contraseña que permite usar un servicio, como un modelo de IA o una base de datos; si queda en el frontend, cualquiera puede verla desde el navegador. Dependencia: paquete de código de terceros que tu proyecto instala. Alucinación de paquetes: el modelo sugiere instalar un paquete inexistente.

ANALOGÍA

Un iceberg: la demo es la punta visible; lo que hunde barcos está debajo de la línea de flotación.

PREGUNTA DE PARTICIPACIÓN

> Si su aplicación manejara datos de clientes, ¿cuál de estos seis riesgos les preocuparía más?

TRANSICIÓN

¿Cómo pasamos, entonces, de un prototipo a algo confiable?

ERRORES CONCEPTUALES QUE EVITAR

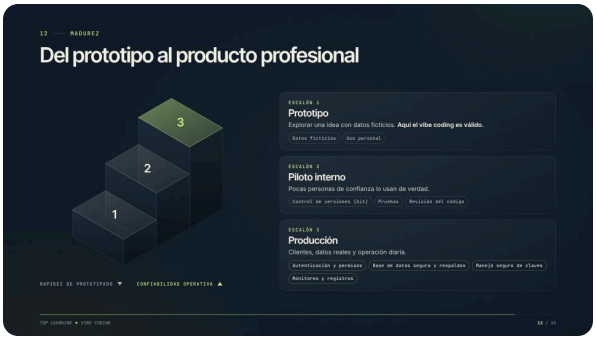
- No digas que el código generado por IA es «inseguro por naturaleza»: el riesgo aparece cuando no se revisa ni se prueba, igual que con el código escrito por personas.
- No cites porcentajes de vulnerabilidades sin una fuente verificada.

RIGOR: HECHO E INTERPRETACIÓN

Hecho documentado: alucinación de paquetes (Spracklen et al., USENIX Security 2025). Las demás categorías son riesgos generales de ingeniería de software, descritos de forma cualitativa.

FUENTES

- Spracklen, J. et al. (2025). «We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs». USENIX Security Symposium. — https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen
- Fowler, M. «Vibe Coding» (bliki). Uso adecuado y riesgos. — https://martinfowler.com/bliki/VibeCoding.html



DIAPOSITIVA 13

De prototipo a producto profesional

1:00

GUION ORAL

Piensen en tres escalones. Prototipo: explorar una idea con datos ficticios; aquí el vibe coding es perfectamente válido. Piloto interno: pocas personas de confianza lo usan de verdad; ya hacen falta control de versiones, pruebas y revisión del código. Producción: clientes, datos reales, operación diaria; aquí se suman autenticación y permisos, una base de datos segura con respaldos, manejo seguro de claves y monitoreo.

Mientras más sube la escalera, más importa la confiabilidad y menos la rapidez. Las dos son valiosas, pero en momentos distintos.

EXPLICACIÓN TÉCNICA

Control de versiones (Git): historial de cambios que permite volver atrás y saber quién cambió qué. Pruebas automatizadas: programas que verifican el comportamiento después de cada cambio. Gestión de secretos: guardar claves fuera del código. Monitoreo y registros (logs): saber cuándo y por qué algo falla.

ANALOGÍA

Cocinar en casa frente a operar un restaurante: en casa, si sale mal, pides pizza; en un restaurante hay normas de higiene, proveedores, inventario y supervisión.

PREGUNTA DE PARTICIPACIÓN

> ¿En qué escalón estaría la aplicación que imaginan para su trabajo?

TRANSICIÓN

Ahora sí, manos a la obra: este es el ejercicio de hoy.

ERRORES CONCEPTUALES QUE EVITAR

- No digas que todo debe llegar a producción: muchos prototipos cumplen su función y se desechan.
- No digas que estas prácticas «ya no hacen falta con IA»: hacen más falta, porque se genera más código, más rápido.

FUENTES

- Fowler, M. «Vibe Coding» (bliki). Uso adecuado y riesgos. — https://martinfowler.com/bliki/VibeCoding.html
- Microsoft. «Best practices for using AI in VS Code». Documentación de Visual Studio Code. — https://code.visualstudio.com/docs/agents/best-practices



DIAPOSITIVA 14

El ejercicio de hoy

1:30

GUION ORAL

Seis pasos. Uno: construir un dashboard financiero con ayuda de IA, usando el prompt de referencia. Dos: crear o usar un Excel ficticio de movimientos. Tres: conectar el archivo a la aplicación mediante el backend. Cuatro: modificar una cifra en Excel. Cinco: guardar el archivo y comprobar que el dashboard se actualiza. Seis: identificar qué indicador cambió y verificar el cálculo a mano.

A la derecha está el resultado de referencia. El suyo no tiene que verse idéntico, pero los números sí deben coincidir: ingresos, 2 millones 750 mil; egresos, 2 millones 38 mil; flujo neto, 712 mil.

Al terminar, cada quien debe poder decir: «cambié esta cifra, este indicador pasó de A a B y lo verifiqué así». Esa es la evidencia de aprendizaje.

EXPLICACIÓN TÉCNICA

Evidencia esperada: captura del dashboard antes y después, la cifra modificada y el cálculo de verificación. El prompt de referencia está en materiales/prompt-referencia.md. Recomendación: usar una herramienta que trabaje en la computadora local, como Claude Code, Codex, Cursor u OpenCode, porque un constructor web en la nube no puede leer un Excel guardado en su equipo.

PREGUNTA DE PARTICIPACIÓN

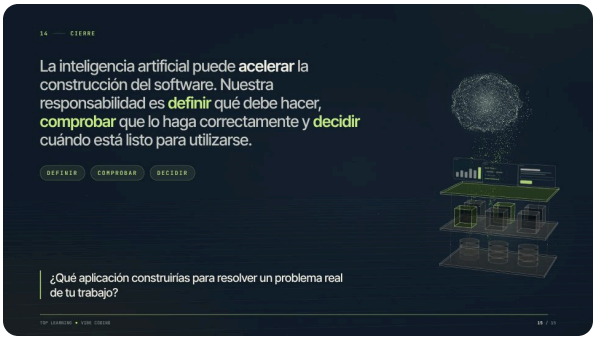
> ¿Qué cifra van a modificar primero y qué indicador esperan que cambie?

TRANSICIÓN

Antes de empezar, cerremos con la idea más importante de hoy.

ERRORES CONCEPTUALES QUE EVITAR

- No pidas usar datos reales de la empresa: solo datos ficticios.
- El ejercicio no termina cuando «se ve bien»: termina cuando el cálculo está verificado.



DIAPOSITIVA 15

Conclusión

1:00

GUION ORAL

Me quedo con una idea: la inteligencia artificial puede acelerar la construcción del software. Nuestra responsabilidad es definir qué debe hacer, comprobar que lo haga correctamente y decidir cuándo está listo para utilizarse.

Definir, comprobar y decidir. Esas tres tareas no se delegan.

Y les dejo la pregunta para la práctica: ¿qué aplicación construirían para resolver un problema real de su trabajo? Escríbanla en el chat; con ella empezamos.

PREGUNTA DE PARTICIPACIÓN

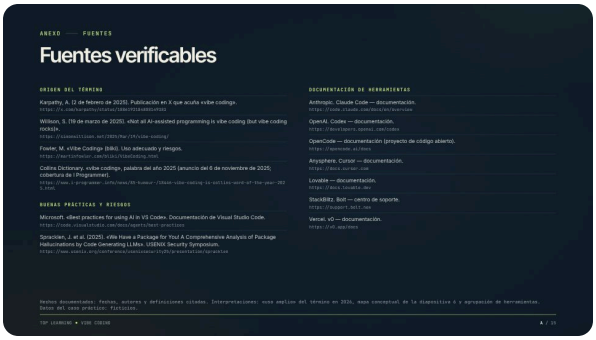
> ¿Qué aplicación construirías para resolver un problema real de tu trabajo?

TRANSICIÓN

Abrir la práctica guiada.

ERRORES CONCEPTUALES QUE EVITAR

- No cierres con predicciones sobre «el fin de la programación» o de los programadores: no están sustentadas.



ANEXO

Anexo · Fuentes

Fuera de los 25 minutos

GUION ORAL

Diapositiva de consulta; no forma parte de los 25 minutos. Muéstrala si alguien pregunta por las referencias o compártela al final junto con el PDF.